JAVASCRIPT

BARDZO ROZBUDOWANA NOTATKA DO NAUKI

Podstawy języka, operatory, sterowanie, pętle, tablice, stringi, funkcje i wyrażenia regularne

Wersja rozszerzona na podstawie tematów z zeszytu i materiałów W3Schools

Listopad 2026

Co znajdziesz w środku?

To opracowanie zachowuje tematy z przesłanego PDF-u, ale rozwija je o praktyczne wyjaśnienia, dodatkowe metody, typowe błędy i przykłady. Nie jest to tylko skrót definicji - ma pomóc zrozumieć, co zapis w kodzie robi i dlaczego.

Jak korzystać z notatki?

Najpierw przeczytaj wyjaśnienie, potem przeanalizuj kod linijka po linijce. Spróbuj przewidzieć wynik, a dopiero później uruchom przykład. Nie ucz się samych nazw metod - zapamiętuj też, co przyjmują i co zwracają.

SPIS TREŚCI

1. JavaScript - co to jest i jak dołączyć skrypt do strony
2. Podstawowa składnia, komentarze, wyświetlanie i okna dialogowe
3. Zmienne: let, const, var, nazwy i zasięg
4. Typy danych, typeof i konwersja typów
5. Operatory w JavaScript
6. Instrukcje warunkowe: if, else i switch
7. Pętle: for, while, do...while, for...of i for...in
8. Tablice - tworzenie, indeksy i odczyt danych
9. Zmiana tablic: push, pop, shift, unshift, splice, slice
10. Przeglądanie i przetwarzanie tablic
11. Sortowanie i odwrócenie tablicy
12. Łańcuchy znaków (String) - zapis i podstawy
13. Metody i właściwości String
14. Znaki, ASCII i Unicode
15. Funkcje - deklarowanie, argumenty i return
16. Rodzaje funkcji, arrow functions, zakres i callbacki
17. Wyrażenia regularne - czym są i jak je zapisujemy
18. RegExp - klasy znaków, kotwice, kwantyfikatory i grupy
19. Flagi i wykorzystywanie RegExp w metodach tekstowych
20. Przykłady złożone i typowe zadania
21. Najczęstsze błędy i szybka powtórka
22. Zestaw ćwiczeń z odpowiedziami
23. Słowniczek najważniejszych pojęć
24. Materiały i źródła

1. JavaScript - co to jest i jak dołączyć skrypt do strony

JavaScript (JS) jest językiem programowania, który najczęściej odpowiada za zachowanie i interaktywność stron internetowych.

1.1. Do czego służy JavaScript?

HTML opisuje strukturę strony, CSS odpowiada za wygląd, a JavaScript pozwala reagować na działania użytkownika i zmieniać zawartość strony. Przykładowo JS może obsłużyć kliknięcie przycisku, sprawdzić formularz, wykonać obliczenie, zmienić tekst albo dodać element do listy.

- HTML - co znajduje się na stronie, np. nagłówek, przycisk, formularz.
- CSS - jak elementy wyglądają, np. kolor, rozmiar, położenie.
- JavaScript - co strona robi, np. co ma się wydarzyć po kliknieciu.

1.2. Sposoby umieszczania kodu JS

Sposób	Na czym polega	Przykład / uwagi
Kod w dokumencie HTML	Kod znajduje się bezpośrednio	Dobre do bardzo malych przykladow;

Sposób	Na czym polega	Przykład / uwagi
	pomiędzy znacznikami script.	przy większej stronie robi się nieczytelnie.
Plik zewnętrzny	Kod znajduje się w osobnym pliku .js.	Najczęściej polecane rozwiązanie. Latwiej utrzymac porzadek i ponownie wykorzystac kod.
Kod inline w atrybucie	Kod jest wpisany np. w onclick elementu HTML.	Da się tak zrobić, ale w większych projektach lepiej oddzielic HTML od JS i obsłużyć zdarzenie w pliku JS.

PRZYKLADY

```
<!-- Kod bezposrednio w HTML -->
<script>
   console.log("Witaj!");
</script>

<!-- Zewnetrzny plik JavaScript -->
<script src="script.js"></script>
```

Atrybut src wskazuje ścieżkę do pliku JavaScript. Jeśli plik znajduje się w tym samym folderze co HTML, wystarczy jego nazwa. Gdy jest w podfolderze, trzeba podac odpowiednia ścieżkę, np. js/script.js.

1.3. Gdzie umiescic znacznik script?

Skrypt może być dolaczony w sekcji head albo w body. Jeżeli umiescimy go wysoko w head bez dodatkowych ustawien, przeglądarka może zatrzymac parsowanie HTML na czas pobierania i wykonywania skryptu. Dlatego często stosuje się atrybut defer albo umieszcza skrypt pod koniec body.

POLECANY ZAPIS Z DEFER

```
<!DOCTYPE html>
<html lang="pl">
<head>
   <meta charset="UTF-8">
   <title>Moja strona</title>
   <script src="script.js" defer></script>
</head>
<body>
   <h1>Witaj!</h1>
</body>
</html>
```

Co robi defer?

Przeglądarka pobiera plik skryptu, ale wykonuje go dopiero po sparsowaniu dokumentu HTML. Przy kilku skryptach z defer zachowana jest kolejność ich wykonania. Atrybut async działa inaczej - skrypt uruchamia się, gdy tylko zostanie pobrany, więc kolejność kilku takich plików nie jest gwarantowana.

1.4. Kilka plików JavaScript na jednej stronie

Można dołączyć kilka plików JS. Warto jednak pamietac, ze kolejność ma znaczenie, jeśli jeden plik korzysta z funkcji lub zmiennych drugiego pliku. Najpierw dolacz plik, który definiuje potrzebne funkcje, a potem plik, który z nich korzysta.

```
<script src="funkcje.js" defer></script>
<script src="strona.js" defer></script>
```

2. Podstawowa składnia, komentarze, wyświetlanie i okna dialogowe

2.1. Wielkość liter i średniki

JavaScript rozroznia wielkie i małe litery. Zmienne tekst i Tekst to dwie różne nazwy. Nazwy metod też trzeba wpisywac dokładnie, np. toUpperCase() jest poprawne, a touppercase() nie.

```
let imie = "Ala";
let Imie = "Ola";

console.log(imie);  // Ala
console.log(Imie);  // Ola
```

Srednik ; kończy instrukcje. W wielu sytuacjach JavaScript potrafi wstawić go automatycznie, ale na początku warto zapisywac średniki konsekwentnie. Nawiasy klamrowe { } wyznaczaja blok kodu, np. cialo funkcji, pętli albo instrukcji if.

2.2. Komentarze

Rodzaj	Zapis	Zastosowanie
Jednoliniowy	// komentarz	Opisuje jedna linie; komputer go nie wykonuje.
Wieloliniowy	/* komentarz dalsza część */	Opisuje dłuższy fragment lub tymczasowo wylacza kilka linii kodu.

```
// Obliczamy sume dwoch liczb
let a = 5;
let b = 7;
let suma = a + b;

/* Ten fragment jest komentarzem.
   Nie zostanie wykonany. */
```

2.3. Wyświetlanie danych - console.log()

console.log() wyświetla wartość w konsoli narzędzi programistycznych przeglądarki. Jest bardzo przydatne do sprawdzania, co zawieraja zmienne i jaki wynik daje obliczenie. Nie jest to to samo co zwracanie wartości przez funkcje (return).

```
let imie = "Hubert";
```

```
let wiek = 17;

console.log(imie);
console.log("Wiek:", wiek);
console.log(5 + 3);
```

Ważne rozroznienie

console.log(wynik) tylko wyświetla wartość w konsoli. return wynik oddaje wartość z funkcji do miejsca, w ktorym funkcja została wywołana. To dwa różne mechanizmy.

2.4. Trzy podstawowe okna dialogowe

Funkcja	Co robi?	Co zwraca?
alert(tekst)	Wyświetla komunikat i przycisk OK.	Nie zwraca przydatnej wartości (undefined).
prompt(pytanie)	Wyświetla pytanie i pole do wpisania tekstu.	String z wpisana wartością albo null, gdy użytkownik anuluje.
confirm(pytanie)	Pyta o potwierdzenie - OK albo Anuluj.	Boolean: true dla OK, false dla Anuluj.

```
alert("Witaj na stronie!");

let imie = prompt("Jak masz na imie?");
console.log(imie);

let zgoda = confirm("Czy kontynuowac?");
console.log(zgoda);
```

Wartość z prompt() jest tekstem, nawet gdy użytkownik wpisze cyfry. Jeżeli chcesz wykonać obliczenie, przekonwertuj ja na liczbę, np. Number(prompt("Podaj liczbę")).

2.5. Document.write()

document.write() zapisuje tekst bezpośrednio do dokumentu HTML. W starych przykladach bywa używane do wyswietlania wyniku, ale w normalnych stronach lepiej unikac tej metody. Wywołana po zaladowaniu strony może nadpisac jej zawartość. Do nauki można użyć console.log(), a do zmiany strony - np. textContent elementu.

```
// Przyklad demonstracyjny
 document.write("Witaj!");
```

3. Zmienne: let, const, var, nazwy i zasięg

Zmienna to nazwana przestrzen, w ktorej program przechowuje wartość. Można ja później odczytac, a w zależności od sposobu deklaracji także zmienić.

3.1. Deklaracja i przypisanie

```
let liczba;      // deklaracja zmiennej
```

```
liczba = 10;      // przypisanie wartosci

let wynik = 5 + 3; // deklaracja i przypisanie
console.log(wynik);
```

Znak = oznacza przypisanie, a nie matematyczna równość. Instrukcja let x = 5 zapisuje wartość 5 w zmiennej x. Do sprawdzania równości służy == albo ===.

3.2. Let, const i var - różnice

Słowo	Czy można zmienić wartość?	Czy można ponownie zadeklarowac w tym samym bloku?	Zakres
let	Tak	Nie	Blok { }
const	Nie można przypisać nowej wartości do tej zmiennej	Nie	Blok { }
var	Tak	Tak	Funkcja, a poza funkcja zakres globalny

W nowoczesnym JavaScript najczęściej używa się const, gdy zmienna nie ma dostac nowego przypisania, i let, gdy wartość będzie zmieniana. var jest starszym sposobem deklarowania i zwykle nie jest polecane w nowym kodzie.

```
const imie = "Ala";
let punkty = 10;

punkty = 15;      // poprawne
// imie = "Ola";   // blad - const nie pozwala na nowe przypisanie
```

Ważna uwaga o const i tablicach

const blokuje ponowne przypisanie zmiennej, ale nie zamraza automatycznie zawartości tablicy lub obiektu. Można dodać element do tablicy zadeklarowanej przez const, ale nie można podmienic calej tablicy na inna.

```
const liczby = [1, 2];
liczby.push(3);      // OK - zawartosc tablicy sie zmienia
// liczby = [4, 5];   // blad - nowe przypisanie
```

3.3. Zasady tworzenia nazw zmiennych

- Nazwa może zawierać litery, cyfry, znak podkreslenia _ i znak dolara $.
- Nie może zaczynac się od cyfry.
- Nie może zawierać spacji ani myślnika jako zwykłego znaku nazwy.
- Wielkość liter ma znaczenie: wynik, Wynik i WYNIK to różne nazwy.
- Nie należy używać słów zarezerwowanych przez JavaScript, np. let, if, return, function.

Poprawna nazwa	Niepoprawna nazwa	Powód
wiek	2wiek	Nie zaczynamy od cyfry.
moja_wartosc	moja wartość	Nie używamy spacji.
$cena	cena-zl	Myślnik nie jest dozwolony w

Poprawna nazwa	Niepoprawna nazwa	Powód
		zwyklej nazwie.
_wynik	let	let jest słowem kluczowym.

Czytelne nazwy często zapisuje się w stylu camelCase, np. liczbaPunktow, imieUzytkownika, sredniaOcen. Nazwa powinna opisywac, co przechowuje zmienna.

3.4. Zakres zmiennych (scope)

Zakres okresla, z jakiego miejsca programu można korzystać ze zmiennej. Najważniejsze są trzy pojecia: zakres globalny, zakres funkcji i zakres bloku.

Zakres	Wyjaśnienie	Przykład
Globalny	Zmienna zadeklarowana poza funkcjami i blokami. Jest dostępna w różnych czesciach programu.	let globalna = 10;
Funkcji	Zmienna zadeklarowana wewnątrz funkcji. Poza funkcja nie można jej bezpośrednio użyć.	function test(){ let lokalna = 5; }
Bloku	let i const zadeklarowane w { } są dostępne tylko w tym bloku.	if(true){ let x = 2; }

```
let globalna = "Widoczna";

function pokaz() {
   let lokalna = "Tylko w funkcji";
   console.log(globalna); // dziala
   console.log(lokalna);  // dziala
}

pokaz();
// console.log(lokalna);   // blad: poza zakresem
```

Dlaczego unikac var?

var nie respektuje zwykłego zakresu bloku. Zmienna zadeklarowana wewnątrz if lub for może być widoczna także poza tym blokiem. let i const zachowują się bardziej przewidywalnie.

4. Typy danych, typeof i konwersja typów

Typ danych okresla, jaki rodzaj wartości jest zapisany w zmiennej i jakie operacje maja dla niej sens.

4.1. Najważniejsze typy danych

Typ	Co przechowuje?	Przykład
Number	Liczby całkowite i ulamkowe.	let cena = 12.50;
String	Tekst, czyli łańcuch znaków.	let imię = "Ala";
Boolean	Wartość logiczna true albo false.	let pełnoletni = false;
Undefined	Zmienna zadeklarowana, ale bez	let wynik;

Typ	Co przechowuje?	Przykład
	przypisanej wartości.	
Null	Celowy brak wartości.	let wybor = null;
BigInt	Bardzo duze liczby całkowite.	let duza = 123n;
Symbol	Unikalny identyfikator.	let id = Symbol("id");
Object	Obiekty, tablice, daty i inne struktury.	let osoba = {imię: "Ala"};

W szkolnych zadaniach najczęściej spotkasz Number, String i Boolean. Tablice oraz obiekty naleza do szerszej kategorii object. BigInt i Symbol warto rozpoznawac, ale na początku zwykle nie są najważniejsze.

4.2. Typ Number

JavaScript używa typu Number zarowno dla liczb całkowitych, jak i ulamkow. Separator dziesietny to kropka, a nie przecinek. Można również użyć zapisu wykladniczego.

```
let a = 12;
let b = 3.14;
let c = 1.5e3;   // 1500
let d = 2e-2;    // 0.02
```

Szczegolnymi wartościami typu Number są NaN, Infinity i -Infinity. NaN oznacza wynik, który nie jest poprawna liczba, np. Number("kot").

```
let a = Number("abc");
console.log(a);            // NaN
console.log(Number.isNaN(a)); // true

console.log(10 / 0);       // Infinity
```

4.3. String i konkatenacja

String to tekst zapisany w cudzyslowach pojedynczych, podwojnych albo w backtickach. Laczenie tekstów operatorem + nazywa się konkatenacja.

```
let imie = "Ala";
let nazwisko = "Kowalska";
let pelne = imie + " " + nazwisko;

console.log(pelne); // Ala Kowalska
```

Uwaga na mieszanie typów

Operator + może łączyć teksty. Dlatego "5" + 2 daje tekst "52", a nie liczbę 7. Przy innych operatorach JS często probuje wykonać konwersje, np. "5" - 2 daje 3. W zadaniach najbezpieczniej jawnie zamieniac dane na odpowiedni typ.

4.4. Boolean - true i false

Boolean ma tylko dwie wartości: true (prawda) i false (fałsz). Wynik porównania, np. 5 > 3, jest wartością logiczna.

```
let a = 5 > 3;     // true
let b = 5 === 3;    // false
let c = Boolean(0); // false
```

4.5. Undefined a null

Wartość	Znaczenie	Przykład
undefined	Wartość nie została ustawiona albo brak wartości został zwrócony przez funkcje.	let x;
null	Programista celowo oznaczyl brak wartości.	let wybranyUzytkownik = null;

4.6. Operator typeof

typeof zwraca tekst okreslajacy typ wartości. Wynikiem nie jest sam typ, lecz napis, np. "number" lub "string".

```
typeof 10;         // "number"
typeof 3.14;       // "number"
typeof "Ala";      // "string"
typeof true;       // "boolean"
typeof undefined;  // "undefined"
typeof {};         // "object"
typeof [];         // "object"
```

Dwa ważne wyjatki

typeof null zwraca "object" - jest to historyczna cecha JavaScript. Tablica także daje typeof === "object". Aby sprawdzić, czy dana wartość jest tablica, użyj Array.isArray(wartość).

4.7. Konwersja typów

Konwersja polega na zamianie wartości z jednego typu na inny. Może odbyc się jawnie, gdy programista wywola odpowiednia funkcje, albo automatycznie w trakcie wykonywania wyrażenia.

Sposób	Działanie	Przykład i wynik
Number()	Zamienia wartość na liczbę.	Number("12") -> 12
String()	Zamienia wartość na tekst.	String(12) -> "12"
Boolean()	Zamienia wartość na true/false.	Boolean(0) -> false
parseInt()	Odczytuje liczbę całkowita z tekstu.	parseInt("12.8") -> 12
parseFloat()	Odczytuje liczbę z czescia ulamkowa.	parseFloat("12.8") -> 12.8

```
let tekst = "25";
let liczba = Number(tekst);
console.log(liczba + 5); // 30

let cena = "12.75 zl";
console.log(parseFloat(cena)); // 12.75
```

Number("12 zl") da NaN, ponieważ caly tekst nie jest poprawnym zapisem liczby. parseFloat("12 zl") odczyta poczatkowe 12. Wybieraj funkcje świadomie.

4.8. Wartości truthy i falsy

W warunku JavaScript może sprawdzac nie tylko true lub false, ale także inna wartość i automatycznie uznac ja za prawdziwa albo fałszywa.

Falsy - traktowane jak false	Przykłady truthy - traktowane jak true
false, 0, -0, 0n	1, -3, 3.14
"" (pusty tekst)	"Ala", a nawet "0"
null, undefined, NaN	[], {}, function(){}

```
let imie = "";
if (imie) {
   console.log("Wpisano imie");
} else {
   console.log("Pole jest puste");
}
```

Nie myl pustego tekstu z tekstem "0"

Pusty tekst "" jest falsy, ale tekst "0" jest truthy. Pusta tablica [] i pusty obiekt {} również są truthy. To często zaskakuje na początku.

5. Operatory w JavaScript

Operator to symbol, który wykonuje działanie na jednej lub kilku wartościach. Wartość, na ktorej działa operator, nazywa się operandem.

5.1. Operatory arytmetyczne

Operator	Nazwa / działanie	Przykład	Wynik
+	Dodawanie	7 + 3	10
-	Odejmowanie	7 - 3	4
*	Mnożenie	7 * 3	21
/	Dzielenie	7 / 2	3.5
%	Reszta z dzielenia	7 % 3	1
**	Potegowanie	2 ** 3	8
++	Zwiększenie o 1	x++	zależy od położenia
--	Zmniejszenie o 1	x--	zależy od położenia

Operator % nie oznacza procentow. Zwraca reszte z dzielenia. Jest przydatny np. do sprawdzania parzystosci: liczba % 2 === 0 oznacza liczbę parzysta.

```
console.log(10 % 3);  // 1
```

```
console.log(12 % 2);  // 0 - liczba parzysta
console.log(2 ** 4);  // 16
```

Inkrementacja i dekrementacja

Inkrementacja ++ zwiększa liczbę o 1, a dekrementacja -- zmniejsza ja o 1. Operator może stac przed zmienna albo za nia.

```
let a = 5;
let wynik1 = ++a; // najpierw zwieksza a, potem przypisuje: wynik1 = 6

let b = 5;
let wynik2 = b++; // najpierw przypisuje stara wartosc: wynik2 = 5,
                  // potem zwieksza b do 6
```

Zapis	Kolejność	Wartość wyrażenia
++x	Najpierw zwiększa x, potem używa wyniku.	Nowa wartość x
x++	Najpierw używa starej wartości, potem zwiększa x.	Stara wartość x
--x	Najpierw zmniejsza x, potem używa wyniku.	Nowa wartość x
x--	Najpierw używa starej wartości, potem zmniejsza x.	Stara wartość x

5.2. Operatory przypisania

Operator przypisania zapisuje wartość w zmiennej. Podstawowym operatorem jest =, a skrocone wersje lacza przypisanie z dzialaniem.

Operator	Odpowiednik	Przykład przy x = 10, y = 3
=	x = y	x otrzyma 3
+=	x = x + y	x otrzyma 13
-=	x = x - y	x otrzyma 7
*=	x = x * y	x otrzyma 30
/=	x = x / y	x otrzyma 3.333...
%=	x = x % y	x otrzyma 1
**=	x = x ** y	x otrzyma 1000

```
let punkty = 10;
punkty += 5;  // 15
punkty -= 2;  // 13
punkty *= 2;  // 26
```

5.3. Operatory porównania

Operatory porównania zestawiaja dwie wartości. Wynikiem jest zawsze true albo false.

Operator	Znaczenie	Przykład
==	Równość po ewentualnej konwersji typów.	5 == "5" -> true
===	Równość wartości i typu (scisla).	5 === "5" -> false
!=	Różne wartości po konwersji.	5 != "6" -> true
!==	Różna wartość lub różny typ.	5 !== "5" -> true
>	Większe niz.	7 > 3 -> true
<	Mniejsze niz.	2 < 1 -> false
>=	Większe lub równe.	5 >= 5 -> true
<=	Mniejsze lub równe.	4 <= 7 -> true

Zwykle wybieraj === i !==

Scisle porównanie nie wykonuje automatycznej konwersji typu. Dlatego jest bardziej przewidywalne. Uzywaj == tylko wtedy, gdy świadomie chcesz porównywać wartości z konwersja.

Teksty również można porównywać, np. "A" < "B". Porównywanie tekstów odbywa się według kolejności kodów znaków, a nie według polskiego słownika. Dla sortowania polskich nazw lepiej wykorzystac localeCompare().

5.4. Operatory logiczne

Operator	Nazwa	Kiedy wynik jest prawdziwy?
&&	AND - i	Gdy oba warunki są prawdziwe.
||	OR - lub	Gdy przynajmniej jeden warunek jest prawdziwy.
!	NOT - negacja	Odwrotnie niz wartość logiczna po prawej stronie.

```
let wiek = 20;
let maBilet = true;

console.log(wiek >= 18 && maBilet); // true
console.log(wiek < 18 || maBilet);  // true
console.log(!maBilet);              // false
```

Tablica prawdy dla AND i OR

A	B	A && B	A || B
false	false	false	false
false	true	false	true
true	false	false	true
true	true	true	true

Operatory && i || działają krotkozwłocznie (short-circuit). Przy && druga część nie musi być sprawdzana, gdy pierwsza jest fałszywa. Przy || druga część nie musi być sprawdzana, gdy pierwsza jest prawdziwa. Dzieki temu można np. zabezpieczyc wykonanie fragmentu kodu.

W JavaScript && i || nie musza zwracac wartości Boolean. Zwracają jedna z wartości operandow. W praktyce najczęściej używa się ich w warunkach, ale warto o tym pamietac przy przypisywaniu wyniku do zmiennej.

```
let imie = "Ala";
let wynik1 = imie && "Witaj " + imie; // "Witaj Ala"
let wynik2 = "" || "Brak danych";  // "Brak danych"
```

5.5. Operator trojargumentowy (ternary)

Operator ? : jest skrócona forma prostego if...else. Najpierw podaje się warunek, po znaku ? wartość dla true, a po : wartość dla false.

```
let wiek = 17;
let komunikat = wiek >= 18 ? "Pelnoletni" : "Niepelnoletni";
console.log(komunikat);
```

Schemat: warunek ? wartosc_gdy_true : wartosc_gdy_false. Operator ternary najlepiej stosowac do prostego wyboru jednej z dwóch wartości. Przy kilku instrukcjach czy wielu warunkach czytelniejszy będzie if...else.

5.6. Operatory bitowe

Operatory bitowe wykonują operacje na bitach. JavaScript przed takim dzialaniem zamienia liczby na 32-bitowe liczby całkowite, a wynik z powrotem przedstawia jako Number. W zwyklych zadaniach webowych są rzadziej potrzebne, ale warto znac ich znaczenie.

Operator	Nazwa	Znaczenie
&	AND bitowy	Bit wyniku jest 1, gdy oba porownywane bity są 1.
|	OR bitowy	Bit wyniku jest 1, gdy przynajmniej jeden z bitow jest 1.
^	XOR	Bit wyniku jest 1, gdy bity są różne.
~	NOT	Odwraca wszystkie bity.
<<	Przesunięcie w lewo	Przesuwa bity w lewo, z prawej strony dopisuje zera.
>>	Przesunięcie w prawo ze znakiem	Przesuwa w prawo, zachowujac bit znaku.
>>>	Przesunięcie w prawo bez znaku	Przesuwa w prawo i z lewej dopisuje zera.

```
// Przyklad na zapisie binarnym:
// 5 = 0101
// 1 = 0001

5 & 1;   // 0001 = 1
5 | 1;   // 0101 = 5
5 ^ 1;   // 0100 = 4
```

```
5 << 1;  // 1010 = 10
5 >> 1;  // 0010 = 2
~5;      // -6 (JavaScript uzywa 32-bitowego zapisu ze znakiem)
```

Bitowe operatory przypisania

Operator	Skrót od	Przykład
&=	x = x & y	x &= y
|=	x = x | y	x |= y
^=	x = x ^ y	x ^= y
<<=	x = x << y	x <<= y
>>=	x = x >> y	x >>= y
>>>=	x = x >>> y	x >>>= y

5.7. Operatory typu

Operator	Co sprawdza?	Przykład
typeof	Zwraca tekst z nazwa typu wartości.	typeof "Ala" -> "string"
instanceof	Sprawdza, czy obiekt jest instancja danego konstruktora / typu.	[] instanceof Array -> true

5.8. Operatory logicznego przypisania

Operator	Działanie
&&=	Przypisuje prawa strone tylko wtedy, gdy lewa zmienna jest truthy.
||=	Przypisuje prawa strone tylko wtedy, gdy lewa zmienna jest falsy.
??=	Przypisuje prawa strone tylko wtedy, gdy lewa wartość jest null albo undefined.

```
let nazwa = "";
nazwa ||= "Domyslna nazwa";
console.log(nazwa);

let wynik = null;
wynik ??= 100;
console.log(wynik); // 100
```

Różnica między || i ??

Operator || wybiera prawa strone dla każdej wartości falsy, np. 0, pustego tekstu, false, null i undefined. Operator ?? robi to tylko dla null i undefined. Dlatego przy liczbie 0 często bezpieczniejszy jest ??.

5.9. Kolejność wykonywania dzialan

Gdy w jednym wyrażeniu występuje kilka operatorow, JavaScript stosuje okreslona kolejność (priorytet). Mnożenie i dzielenie wykonuje się przed dodawaniem i odejmowaniem. Nawiasy pozwalaja jasno wskazac, co ma być policzone jako pierwsze.

```
let a = 2 + 3 * 4;     // 14
let b = (2 + 3) * 4;   // 20
```

Dobra praktyka

Gdy nie masz pewnosci co do kolejności dzialan, użyj nawiasów. Kod staje się czytelniejszy i trudniej o pomylke.

6. Instrukcje warunkowe: if, else i switch

Instrukcje warunkowe pozwalaja programowi wybrać, który fragment kodu ma zostać wykonany.

6.1. Instrukcja if

if wykonuje blok kodu wtedy, gdy warunek jest prawdziwy (true). Warunek zapisujemy w nawiasach okrągłych, a wykonywane instrukcje w nawiasach klamrowych.

```
let temperatura = 25;

if (temperatura > 20) {
   console.log("Jest cieplo");
}
```

6.2. If...else

else okresla, co ma się wydarzyć, gdy warunek if jest fałszywy.

```
let liczba = 7;

if (liczba % 2 === 0) {
   console.log("Liczba parzysta");
} else {
   console.log("Liczba nieparzysta");
}
```

6.3. Else if - kilka warunkow

else if pozwala sprawdzić kolejny warunek, jeśli poprzedni nie został spełniony. Program wykonuje pierwszy blok, ktorego warunek jest prawdziwy, a potem pomija pozostale galezie.

```
let punkty = 82;

if (punkty >= 90) {
   console.log("Bardzo dobry");
} else if (punkty >= 75) {
   console.log("Dobry");
} else if (punkty >= 50) {
   console.log("Dostateczny");
```

```
} else {
   console.log("Do poprawy");
}
```

Kolejność warunkow ma znaczenie

Najpierw sprawdzaj warunki bardziej szczegolowe albo wymagajace wyzszej wartości. Gdyby w przykladzie jako pierwszy warunek byl punkty >= 50, wynik 82 od razu zostalby zakwalifikowany do tej galezi i program nie sprawdzilby wyzszych progow.

6.4. Warunki laczone

Warunki można łączyć operatorami && (i), || (lub) oraz ! (negacja).

```
let wiek = 19;
let maDokument = true;

if (wiek >= 18 && maDokument) {
   console.log("Warunki spelnione");
} else {
   console.log("Brak spelnionych warunkow");
}
```

6.5. Instrukcja switch

switch służy do wyboru jednego z kilku blokow na podstawie wartości wyrażenia. Każdy wariant zaczyna się od case, a default jest wariantem domyslnym, wykonywanym, gdy zaden case nie pasuje.

```
let dzien = 3;

switch (dzien) {
   case 1:
      console.log("Poniedzialek");
      break;
   case 2:
      console.log("Wtorek");
      break;
   case 3:
      console.log("Sroda");
      break;
   default:
      console.log("Nieznany dzien");
}
```

Po co jest break?

break kończy wykonywanie instrukcji switch. Gdy go zabraknie, program przejdzie do kolejnego case, nawet jeśli jego wartość nie pasuje. Takie przechodzenie nazywa się fall-through. Czasem jest celowe, np. gdy kilka wartości ma wykonywac ten sam kod.

```
let dzien = 6;

switch (dzien) {
   case 6:
   case 7:
      console.log("Weekend");
```

```
    break;
  default:
    console.log("Dzien roboczy");
}
```

switch korzysta ze scislego porównania

W instrukcji switch wartość jest porownywana z case tak jak przy ===. Liczba 2 i tekst "2" nie są tym samym przypadkiem.

6.6. Kiedy if, a kiedy switch?

Wybierz if...else, gdy:	Wybierz switch, gdy:
Sprawdzasz zakres, np. wiek >= 18.	Porownujesz jedna zmienna z wieloma konkretnymi wartościami.
Laczysz kilka warunkow operatorami && lub ||.	Masz wiele wariantow dla jednej wartości, np. kod menu.
Warunki są różne i bardziej rozbudowane.	Kod dla case jest krótki i latwy do uporzadkowania.

7. Pętle: for, while, do...while, for...of i for...in

Pętla pozwala wielokrotnie wykonywac ten sam fragment kodu. Przy każdej pętli trzeba uważać, aby warunek kiedys przestal być prawdziwy - inaczej powstanie pętla nieskończona.

7.1. Pętla for

for jest wygodna, gdy zwykle z gory wiadomo, ile razy pętla ma się wykonać. Składa się z trzech części: inicjalizacji, warunku i zmiany licznika.

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

Część	Przykład	Znaczenie
Inicjalizacja	let i = 0	Ustawia licznik na początku.
Warunek	i < 5	Pętla wykonuje się, dopoki warunek jest true.
Zmiana	i++	Po każdym obiegu zwiększa licznik o 1.

Wynik przykladu to liczby 0, 1, 2, 3, 4. Gdy i stanie się równe 5, warunek i < 5 będzie fałszywy i pętla się zakonczy.

Odliczanie w dol i przeskok co 2

```
// Od 5 do 1
for (let i = 5; i >= 1; i--) {
  console.log(i);
```

```
}

// Liczby parzyste od 0 do 10
for (let i = 0; i <= 10; i += 2) {
   console.log(i);
}
```

7.2. Pętla while

while sprawdza warunek przed wykonaniem bloku. Jeżeli warunek od początku jest fałszywy, cialo pętli nie wykona się ani razu.

```
let i = 0;

while (i < 5) {
   console.log(i);
   i++;
}
```

Najczestszy błąd w while

Zapomnienie o zmianie zmiennej, od ktorej zależy warunek. Jeśli i pozostanie stale równe 0, warunek i < 5 nigdy się nie zmieni i pętla będzie dzialala bez końca.

7.3. Pętla do...while

do...while najpierw wykonuje blok, a dopiero potem sprawdza warunek. Dlatego wykona się przynajmniej raz, nawet gdy warunek jest fałszywy.

```
let liczba = 10;

do {
   console.log("Wykonano petle");
} while (liczba < 5);
```

7.4. Porównanie pętli

Pętla	Kiedy sprawdza warunek?	Czy może nie wykonać się ani razu?
for	Przed każdym obiegiem.	Tak.
while	Przed każdym obiegiem.	Tak.
do...while	Po wykonaniu bloku.	Nie - wykona się przynajmniej raz.

7.5. Break i continue

Instrukcja	Działanie	Przykład zastosowania
break	Natychmiast kończy cala pętle.	Znaleziono szukany element i dalsze sprawdzanie nie jest potrzebne.
continue	Pomija reszte aktualnego obiegu i	Nie wykonuj obliczen dla liczb

Instrukcja	Działanie	Przykład zastosowania
	przechodzi do następnego.	nieparzystych.

```
// break - petla zatrzyma sie przy 3
for (let i = 0; i < 6; i++) {
   if (i === 3) break;
   console.log(i);
}

// continue - pomin liczbe 3
for (let i = 0; i < 6; i++) {
   if (i === 3) continue;
   console.log(i);
}
```

7.6. For...of - przechodzenie po wartościach

for...of pobiera kolejne wartości z elementow tablicy lub innego obiektu iterowalnego. Nie trzeba samodzielnie obsługiwać indeksu.

```
const owoce = ["jablko", "gruszka", "banan"];

for (const owoc of owoce) {
   console.log(owoc);
}
```

7.7. For...in - przechodzenie po kluczach

for...in przechodzi po nazwach właściwości obiektu. Przy tablicy otrzymasz zwykle jej indeksy zapisane jako tekst. W przypadku zwyklych tablic czesciej poleca się klasyczne for, for...of albo metody tablicowe.

```
const owoce = ["jablko", "gruszka", "banan"];

for (const indeks in owoce) {
   console.log(indeks, owoce[indeks]);
}
```

Zapamietaj różnice

for...of daje wartości elementow. for...in daje klucze / indeksy. Przy tablicy owoce[1] oznacza element pod indeksem 1, a nie element numer 1 liczony od jedynki.

8. Tablice - tworzenie, indeksy i odczyt danych

Tablica (Array) przechowuje wiele wartości pod jedna nazwa. Każdy element ma swoj indeks, czyli numer pozycji.

8.1. Tworzenie tablicy

```
const owoce = ["jablko", "gruszka", "banan"];
const liczby = [10, 20, 30, 40];
```

```
const rozne = [7, "tekst", true, null];
const pusta = [];
```

Elementy tablicy oddzielamy przecinkami i zapisujemy w nawiasach kwadratowych. Tablica może przechowywac różne typy danych, chociaz w praktyce często wygodniej jest trzymac w jednej tablicy wartości tego samego rodzaju.

8.2. Indeksy tablicy

Indeksowanie zaczyna się od zera. Pierwszy element ma indeks 0, drugi indeks 1 itd. Do odczytu używamy nazwy tablicy i indeksu w nawiasach kwadratowych.

```
const owoce = ["jablko", "gruszka", "banan"];

console.log(owoce[0]); // jablko
console.log(owoce[1]); // gruszka
console.log(owoce[2]); // banan
```

Indeks	Wartość	Znaczenie
0	jablko	Pierwszy element
1	gruszka	Drugi element
2	banan	Trzeci element

8.3. Właściwość length

length zwraca liczbę elementow tablicy. Nie jest metoda, dlatego zapisujemy ja bez nawiasów okrągłych.

```
const liczby = [10, 20, 30, 40];
console.log(liczby.length); // 4
console.log(liczby[liczby.length - 1]); // 40 - ostatni element
```

Ostatni element ma indeks length - 1, ponieważ pierwszy indeks to 0. Odwołanie do nieistniejacego indeksu zwykle zwraca undefined.

8.4. Zmiana elementu

```
const oceny = [2, 3, 4];
oceny[0] = 5;
console.log(oceny); // [5, 3, 4]
```

Tablica zadeklarowana przez const nadal może zmieniać swoje elementy. const blokuje przypisanie nowej tablicy do tej samej zmiennej, ale nie blokuje modyfikacji jej zawartości.

8.5. Tablice wielowymiarowe

Tablica może przechowywac inne tablice. W ten sposób można zapisać np. tabele z wierszami i kolumnami. Wtedy do elementu odwołujemy się przez dwa indeksy.

```
const plansza = [
  [1, 2, 3],
```

```
    [4, 5, 6],
    [7, 8, 9]
];

console.log(plansza[0]);    // [1, 2, 3]
console.log(plansza[0][1]); // 2
console.log(plansza[2][2]); // 9
```

9. Zmiana tablic: push, pop, shift, unshift, splice, slice

Metody tablicowe pozwalaja dodawac, usuwac, kopiowac i łączyć elementy. Trzeba rozrozniac metody, które zmieniaja oryginalna tablice, od tych, które tworza nowa.

9.1. Dodawanie i usuwanie na koncu

Metoda	Co robi?	Co zwraca?	Czy zmienia tablice?
push(x)	Dodaje element lub elementy na koncu.	Nowa długość tablicy.	Tak.
pop()	Usuwa ostatni element.	Usuniety element.	Tak.

```
const liczby = [1, 2, 3];

let nowaDlugosc = liczby.push(4);
console.log(liczby);      // [1, 2, 3, 4]
console.log(nowaDlugosc);  // 4

let usunieta = liczby.pop();
console.log(usunieta);    // 4
console.log(liczby);      // [1, 2, 3]
```

9.2. Dodawanie i usuwanie na początku

Metoda	Co robi?	Co zwraca?	Czy zmienia tablice?
unshift(x)	Dodaje element lub elementy na początku.	Nowa długość tablicy.	Tak.
shift()	Usuwa pierwszy element.	Usuniety element.	Tak.

```
const owoce = ["gruszka", "banan"];
owoce.unshift("jablko");
console.log(owoce); // ["jablko", "gruszka", "banan"]

let pierwszy = owoce.shift();
console.log(pierwszy); // "jablko"
```

9.3. Splice() - dodawanie i usuwanie w środku

splice() może usuwac elementy, wstawiać nowe elementy albo jednocześnie usuwac i wstawiać. Jest metoda zmieniająca oryginalna tablice.

```
tablica.splice(indeksStartowy, liczbaDoUsuniecia, element1, element2, ...);
```

Argument	Znaczenie
indeksStartowy	Indeks miejsca, w ktorym ma rozpocząć się operacja.
liczbaDoUsuniecia	Ile elementow usunac od podanego indeksu. 0 oznacza brak usuwania.
element1, element2...	Opcjonalne wartości, które maja zostać wstawione w to miejsce.

```
const liczby = [10, 20, 30, 40];

// Od indeksu 1 usun 0 elementow i wstaw 15, 18
liczby.splice(1, 0, 15, 18);
console.log(liczby); // [10, 15, 18, 20, 30, 40]

// Od indeksu 3 usun 2 elementy
let usuniete = liczby.splice(3, 2);
console.log(usuniete); // [20, 30]
console.log(liczby);   // [10, 15, 18, 40]
```

splice() zwraca tablice elementow usunietych z oryginalnej tablicy. Gdy niczego nie usunieto, zwróci pusta tablice.

9.4. Slice() - wycinanie fragmentu bez zmiany oryginalu

slice() tworzy nowa tablice zawierajaca wybrany fragment. Pierwszy argument to indeks początkowy, drugi to indeks końcowy, ale element z indeksu końcowego nie jest dolaczany.

```
const liczby = [10, 20, 30, 40, 50];
const fragment = liczby.slice(1, 4);

console.log(fragment); // [20, 30, 40]
console.log(liczby);   // [10, 20, 30, 40, 50]
```

Splice a slice - nie pomyl!

splice() zmienia oryginalna tablice i służy do wstawiania/usuwania. slice() tworzy nowa tablice z fragmentem i nie zmienia oryginalu. Roznia się jedna litera, ale działają inaczej.

9.5. Inne przydatne metody tablicowe

Metoda / właściwość	Działanie	Przykład
length	Zwraca liczbę elementow.	tablica.length
concat()	Łączy tablice i zwraca nowa tablice.	a.concat(b)
includes()	Sprawdza, czy element występuje.	liczby.includes(20)
indexOf()	Zwraca indeks pierwszego wystąpienia lub -1.	liczby.indexOf(20)
lastIndexOf()	Zwraca indeks ostatniego wystąpienia lub -1.	liczby.lastIndexOf(20)
join()	Łączy elementy w jeden tekst z separatorem.	owoce.join(", ")

Metoda / właściwość	Działanie	Przykład
toString()	Zamienia tablice na tekst rozdzielony przecinkami.	liczby.toString()
Array.isArray()	Sprawdza, czy wartość jest tablica.	Array.isArray(liczby)

9.6. Dlaczego delete nie jest dobrym sposobem usuwania?

delete tablica[indeks] usuwa wartość z danego miejsca, ale nie przesuwa pozostalych elementow i nie zmniejsza length. W tablicy pozostaje puste miejsce. Do zwykłego usuwania elementow lepiej wykorzystac pop(), shift() albo splice().

10. Przeglądanie i przetwarzanie tablic

Elementy tablicy można przetwarzac klasyczna pętla for, for...of albo metodami tablicowymi. Metody takie jak forEach(), map(), filter() i reduce() przyjmują funkcje, która jest uruchamiana dla kolejnych elementow. Taka przekazana funkcja nazywa się funkcja zwrotna (callback).

10.1. Klasyczna pętla for

```
const liczby = [2, 4, 6];
let suma = 0;

for (let i = 0; i < liczby.length; i++) {
   suma += liczby[i];
}

console.log(suma); // 12
```

W każdym obiegu i jest kolejnym indeksem. liczby[i] pobiera wartość spod tego indeksu. suma += liczby[i] dodaje wartość do zmiennej suma.

10.2. ForEach() - wykonanie operacji dla kazdego elementu

forEach() uruchamia podana funkcje dla kazdego elementu tablicy. Najczęściej do funkcji przekazuje wartość elementu, indeks oraz cala tablice. forEach() sam z siebie nie tworzy nowej tablicy i zwraca undefined.

```
const liczby = [2, 4, 6];

liczby.forEach(function(element, indeks) {
   console.log(indeks, element);
});

// Ten sam pomysl krocej - funkcja strzalkowa
liczby.forEach((element) => console.log(element));
```

10.3. Map() - utworzenie nowej tablicy po przeksztalceniu

map() przechodzi po elementach i tworzy nowa tablice z wynikami funkcji. Dla kazdego elementu funkcja powinna zwrócić wartość, która trafi do nowej tablicy.

```
const liczby = [1, 2, 3, 4];
const podwojone = liczby.map((liczba) => liczba * 2);
```

```
console.log(podwojone); // [2, 4, 6, 8]
console.log(liczby);    // [1, 2, 3, 4]
```

10.4. Filter() - wybieranie elementow spelniajacych warunek

filter() tworzy nowa tablice z tymi elementami, dla których funkcja zwrocila true. Jest przydatne do wybierania np. liczb parzystych, osob pelnoletnich albo produktow o odpowiedniej cenie.

```
const liczby = [1, 2, 3, 4, 5, 6];
const parzyste = liczby.filter((liczba) => liczba % 2 === 0);

console.log(parzyste); // [2, 4, 6]
```

10.5. Reduce() - sprowadzenie tablicy do jednej wartości

reduce() przetwarza elementy po kolei i łączy je w jeden wynik, np. sume, iloczyn albo obiekt podsumowujacy. Funkcja otrzymuje wartość zbierana (akumulator) i aktualny element. Drugi argument reduce() jest wartością początkowa akumulatora.

```
const liczby = [2, 4, 6];
const suma = liczby.reduce((akumulator, liczba) => {
   return akumulator + liczba;
}, 0);

console.log(suma); // 12
```

Przebieg: akumulator zaczyna od 0. Po dodaniu 2 ma wartość 2, potem po dodaniu 4 ma 6, a po dodaniu 6 ma 12. Końcowa wartość jest wynikiem reduce().

10.6. Find(), some() i every()

Metoda	Co zwraca?	Przykład
find()	Pierwszy element spelniajacy warunek albo undefined.	liczby.find(x => x > 5)
some()	true, gdy przynajmniej jeden element spełnia warunek.	liczby.some(x => x < 0)
every()	true, gdy wszystkie elementy spelniaja warunek.	liczby.every(x => x > 0)

```
const oceny = [3, 4, 5, 2];

console.log(oceny.find(x => x < 3));   // 2
console.log(oceny.some(x => x === 1)); // false
console.log(oceny.every(x => x >= 2)); // true
```

10.7. Która metode wybrać?

Cel	Metoda	Wynik
Wyświetlić każdy element / wykonać efekt	forEach()	Nie tworzy tablicy wynikowej.

Cel	Metoda	Wynik
uboczny.		
Zmienić każdy element na nowa wartość.	map()	Nowa tablica o tej samej liczbie elementow.
Wybrać tylko elementy pasujace do warunku.	filter()	Nowa, zwykle krotsza tablica.
Policzyc jedna wartość, np. sume.	reduce()	Jedna wartość.
Znalezc pierwszy pasujacy element.	find()	Element albo undefined.
Sprawdzić, czy chociaz jeden pasuje.	some()	true albo false.
Sprawdzić, czy wszystkie pasuja.	every()	true albo false.

Wskazowka do nauki

forEach() nie jest zamiennikiem map(). Jeśli chcesz utworzyc nowa tablice z wynikami - użyj map(). Jeśli chcesz tylko wykonać operacje dla kazdego elementu - forEach() będzie czytelniejsze.

11. Sortowanie i odwrócenie tablicy

11.1. Sort() - sortowanie tekstów

sort() domyślnie sortuje elementy jako teksty według kolejności znaków. Dla napisow typu nazwy miesięcy czy imiona może to być oczekiwane zachowanie.

```
const owoce = ["banan", "jablko", "gruszka"];
owoce.sort();
console.log(owoce);
```

11.2. Sortowanie liczb - konieczna funkcja porównująca

Domyślne sortowanie może dac zly wynik dla liczb, ponieważ porownuje ich tekstowe postacie. Na przykład tekst "100" może być ustawiony przed "25". Aby sortowac liczby matematycznie, podaj funkcje porównująca.

```
const liczby = [40, 100, 1, 5, 25, 10];

liczby.sort((a, b) => a - b); // rosnaco
console.log(liczby);

liczby.sort((a, b) => b - a); // malejaco
console.log(liczby);
```

Wynik a - b	Co oznacza?	Kolejność
Wynik ujemny	a powinno być przed b.	Rosnąco.
Wynik dodatni	b powinno być przed a.	Malejąco w zależności od funkcji.
Wynik 0	Porownywane wartości są równe według kryterium.	Nie trzeba zmieniać ich wzajemnej kolejności.

11.3. Reverse() i kopie tablicy

reverse() odwraca kolejność elementow w oryginalnej tablicy. Metoda zmienia tablice, na ktorej została wywołana.

```
const liczby = [1, 2, 3];
liczby.reverse();
console.log(liczby); // [3, 2, 1]
```

W nowszym JavaScript dostępne są toSorted() i toReversed(), które tworza nowa tablice zamiast zmieniać oryginal. Podobna zasada dotyczy toSpliced() w porownaniu ze splice().

```
const liczby = [3, 1, 2];
const posortowane = liczby.toSorted((a, b) => a - b);
const odwrocone = liczby.toReversed();

console.log(liczby);     // [3, 1, 2]
console.log(posortowane); // [1, 2, 3]
console.log(odwrocone);   // [2, 1, 3]
```

Sortowanie polskich wyrazów

Dla nazw zawierajacych polskie znaki mozesz użyć funkcji localeCompare(), np. nazwy.sort((a, b) => a.localeCompare(b, "pl")). To uwzględnia reguly porzadkowania danego języka.

12. Łańcuchy znaków (String) - zapis i podstawy

String jest typem danych przechowujacym tekst: litery, cyfry, spacje i inne znaki.

12.1. Trzy sposoby zapisu tekstu

Rodzaj zapisu	Przykład	Co warto wiedziec?
Cudzysłów podwójny	"Ala ma kota"	Najczęściej stosowany sposób.
Apostrof	'Ala ma kota'	Działa tak samo jak cudzysłów podwójny.
Backtick	`Ala ma kota`	Pozwala na wstawianie zmiennych przez ${...} i tekst wieloliniowy.

```
let tekst1 = "To jest tekst";
let tekst2 = 'To tez jest tekst';

let imie = "Ala";
let tekst3 = `Czesc, ${imie}!`;

console.log(tekst3); // Czesc, Ala!
```

12.2. Apostrofy i cudzysłowy wewnątrz tekstu

Można umieszczac apostrof w tekście ujetym w cudzysłowy podwojne i odwrotnie. Gdy znak jest taki sam jak znak ograniczajacy tekst, trzeba go poprzedzic znakiem ucieczki (backslash).

```
let a = "To jest 'cytat'";
```

```
let b = 'Powiedzial: "Czesc!"';
let c = "Powiedzial: \"Czesc!\"";
```

12.3. Sekwencje ucieczki

Sekwencje ucieczki pozwalaja zapisać znaki specjalne wewnątrz łańcucha znaków.

Zapis	Znaczenie	Przykład
\n	Nowy wiersz.	"A\nB"
\t	Tabulacja pozioma.	"A\tB"
\r	Powrót karetki.	"tekst\r"
\b	Backspace.	"A\bB"
\f	Form feed / nowa strona.	"A\fB"
\v	Tabulacja pionowa.	"A\vB"
\'	Apostrof.	'It\'s OK'
\"	Cudzysłów podwójny.	"Powiedział: \"OK\""
\\	Znak backslash.	"C:\\folder"

```
let tekst = "Pierwsza linia\nDruga linia";
console.log(tekst);

let sciezka = "C:\\Users\\Student";
console.log(sciezka);
```

HTML   a JavaScript \n

  to encja HTML oznaczajaca nielamliwa spacje. Nie jest sekwencja ucieczki JavaScript. Z kolei \n jest sekwencja JS oznaczajaca nowy wiersz w tekście.

12.4. Template literals - tekst z wstawianymi zmiennymi

Backticki umożliwiają wstawianie wartości zmiennych do tekstu bez łączenia wielu fragmentow operatorem +. Taki zapis jest czytelniejszy, gdy komunikat zawiera kilka wartości.

```
let imie = "Hubert";
let punkty = 85;

let komunikat = `Uczen ${imie} zdobyl ${punkty} punktow.`;
console.log(komunikat);
```

Template literal może też zajmowac kilka wierszy kodu, a znaki nowej linii zostana zachowane w tekście.

13. Metody i właściwości String

Łańcuchy znaków maja właściwości i metody. Właściwość length odczytujemy bez nawiasów, natomiast metode wywołujemy z nawiasami, np. toUpperCase().

13.1. Najważniejsze metody String

Metoda / właściwość	Co robi?	Przykład
length	Zwraca długość tekstu.	tekst.length
at(i)	Zwraca znak o podanej pozycji; dopuszcza indeks ujemny.	tekst.at(-1)
charAt(i)	Zwraca znak pod indeksem.	tekst.charAt(0)
charCodeAt(i)	Zwraca kod UTF-16 znaku pod indeksem.	tekst.charCodeAt(0)
toUpperCase()	Zamienia litery na wielkie.	tekst.toUpperCase()
toLowerCase()	Zamienia litery na małe.	tekst.toLowerCase()
trim()	Usuwa białe znaki z początku i końca.	tekst.trim()
includes(x)	Sprawdza, czy tekst zawiera fragment.	tekst.includes("kot")
startsWith(x)	Sprawdza, czy tekst zaczyna się od fragmentu.	tekst.startsWith("A")
endsWith(x)	Sprawdza, czy tekst kończy się fragmentem.	tekst.endsWith("!")
indexOf(x)	Indeks pierwszego wystąpienia lub -1.	tekst.indexOf("a")
lastIndexOf(x)	Indeks ostatniego wystąpienia lub -1.	tekst.lastIndexOf("a")
slice(a, b)	Wycina fragment; indeks końcowy nie jest wlaczony.	tekst.slice(1, 4)
substring(a, b)	Wycina fragment; wartości ujemne traktuje jak 0.	tekst.substring(1, 4)
replace(a, b)	Zamienia pierwsze dopasowanie tekstu lub wzorca.	tekst.replace("kot", "pies")
replaceAll(a, b)	Zamienia wszystkie wystąpienia tekstu.	tekst.replaceAll("a", "o")
split(separator)	Dzieli tekst na tablice.	tekst.split(" ")
repeat(n)	Powtarza tekst n razy.	"ha".repeat(3)
padStart(n, x)	Dopełnia tekst od początku.	"5".padStart(3, "0")
padEnd(n, x)	Dopełnia tekst od końca.	"5".padEnd(3, "0")

13.2. Odczyt pojedynczego znaku

```
let tekst = "KOT";

console.log(tekst[0]);       // K
console.log(tekst.charAt(1)); // O
console.log(tekst.at(-1));    // T
```

Indeksy rozpoczynaja się od 0. at(-1) oznacza ostatni znak, at(-2) przedostatni itd. Zapis tekst[0] jest podobny do odczytu elementu tablicy, ale łańcuch znaków nie jest tablica.

13.3. Wyszukiwanie fragmentow tekstu

```
let tekst = "JavaScript jest przydatny";

console.log(tekst.includes("Script")); // true
console.log(tekst.startsWith("Java")); // true
console.log(tekst.endsWith("ny"));     // true
console.log(tekst.indexOf("jest"));    // indeks poczatku wyrazu
```

Metody includes(), startsWith() i endsWith() zwracają Boolean. indexOf() zwraca numer indeksu pierwszego dopasowania albo -1, gdy szukanego tekstu nie ma.

Wielkość liter ma znaczenie

Wyszukiwanie tekstu metodami includes(), startsWith() i indexOf() jest domyślnie wrazliwe na wielkie i małe litery. Na przykład tekst.includes("kot") nie znajdzie fragmentu "Kot". Jeśli chcesz ignorowac wielkość liter, mozesz najpierw użyć toLowerCase() na obu tekstach.

13.4. Wycinanie fragmentow - slice i substring

```
let tekst = "Programowanie";

console.log(tekst.slice(0, 7));     // Program
console.log(tekst.slice(7));        // owanie
console.log(tekst.slice(-4));       // wanie
console.log(tekst.substring(0, 7)); // Program
```

W slice(start, end) indeks end jest pomijany. slice() obsługuje indeksy ujemne liczone od końca. substring() traktuje ujemne wartości jako 0 i zamienia argumenty miejscami, jeżeli pierwszy jest większy od drugiego.

13.5. Split() - zamiana tekstu na tablice

split() dzieli łańcuch znaków według podanego separatora. To bardzo przydatna metoda, gdy mamy tekst z danymi rozdzielonymi spacjami, przecinkami albo innym znakiem.

```
let zdanie = "Ala ma kota";
let wyrazy = zdanie.split(" ");
console.log(wyrazy); // ["Ala", "ma", "kota"]

let dane = "Jan;Kowalski;17";
let pola = dane.split(";");
console.log(pola[0]); // Jan
```

13.6. Replace() i replaceAll()

```
let tekst = "kot i kot";

let pierwszy = tekst.replace("kot", "pies");
console.log(pierwszy); // pies i kot

let wszystkie = tekst.replaceAll("kot", "pies");
console.log(wszystkie); // pies i pies
```

String jest niemutowalny

Metody tekstowe nie zmieniaja oryginalnego stringa. Zwracają nowy tekst, więc jeśli chcesz zachować wynik, przypisz go do zmiennej: tekst = tekst.toUpperCase().

14. Znaki, ASCII i Unicode

14.1. Kodowanie znaków

Komputer przechowuje tekst jako liczby-kody znaków. ASCII to historyczny zestaw kodów dla podstawowych znaków, m.in. liter alfabetu angielskiego, cyfr i znaków sterujacych. Standardowe ASCII obejmuje kody 0-127. Zakres 128-255 bywa nazywany rozszerzonym ASCII, ale jego znaczenie zależy od konkretnego kodowania.

Wspolczesny tekst na stronach internetowych korzysta z Unicode, który pozwala opisywac znaki wielu języków, w tym polskie litery, oraz wiele innych symboli. JavaScript przechowuje stringi w jednostkach kodowych UTF-16.

14.2. FromCharCode() i charCodeAt()

Metoda	Kierunek	Co robi?
String.fromCharCode(kod)	Kod -> znak	Tworzy tekst z podanych jednostek kodowych UTF-16.
tekst.charCodeAt(indeks)	Znak -> kod	Zwraca jednostke kodowa UTF-16 znaku pod indeksem.
tekst.codePointAt(indeks)	Znak -> punkt kodowy	Zwraca punkt kodowy Unicode, także dla znaków spoza podstawowej plaszczyzny.

```
let znak = String.fromCharCode(65);
console.log(znak); // A

let kod = "A".charCodeAt(0);
console.log(kod);  // 65
```

14.3. Przykład - wypisanie kodów i znaków

```
for (let kod = 65; kod <= 90; kod++) {
   console.log(kod, String.fromCharCode(kod));
}
```

Kod od 65 do 90 odpowiada wielkim literom alfabetu lacinskiego A-Z. Podobnie można wygenerować małe litery, cyfry i inne zakresy znaków.

14.4. Generowanie losowego tekstu z zakresu kodów

```
let dlugosc = 10;
let min = 65;
let max = 90;
let tekst = "";

for (let i = 0; i < dlugosc; i++) {
   let kod = Math.floor(Math.random() * (max - min + 1)) + min;
   tekst += String.fromCharCode(kod);
}

console.log(tekst);
```

Wzór Math.floor(Math.random() * (max - min + 1)) + min losuje liczbę całkowita z zakresu od min do max wlacznie. W przykladzie powstaje tekst o zadanej długości z wielkich liter A-Z.

Nie traktuj zakresu 0-255 jako calego Unicode

W starszych notatkach pojawia się zakres 0-255. Jest przydatny do prostych ćwiczeń z kodami znaków, ale nie obejmuje wszystkich liter narodowych i symboli. Dla pelniejszej obslugi Unicode istnieja m.in. codePointAt() i String.fromCodePoint().

14.5. Sortowanie liter w tekście

Metoda sort() należy do tablic, a nie do zwykłego stringa. Aby posortować litery tekstu, trzeba najpierw zamienic go na tablice znaków, posortować tablice i polaczyc wynik z powrotem.

```
let tekst = "KOT";
let posortowany = tekst.split("").sort().join("");
console.log(posortowany);
```

Przy literach z polskimi znakami kolejność sortowania może wymagac localeCompare("pl"). Prosty sort() nie zawsze ulozy polskie litery tak, jak oczekujemy w slowniku.

15. Funkcje - deklarowanie, argumenty i return

Funkcja to wydzielony fragment kodu, który można uruchamiac wielokrotnie. Pozwala unikac powtarzania tego samego kodu i dzielic program na mniejsze części.

15.1. Funkcja bezargumentowa

Funkcja bezargumentowa nie przyjmuje danych z zewnatrz. W nawiasach okrągłych nie ma parametrów.

```
function powitaj() {
   console.log("Witaj!");
}

powitaj(); // wywolanie funkcji
```

15.2. Funkcja z parametrami

Parametry są nazwami wpisanymi w definicji funkcji. Otrzymuja wartości przekazane podczas wywołania.

```
function powitaj(imie) {
   console.log("Czesc " + imie);
}

powitaj("Ala");
powitaj("Jan");
```

Pojecie	Znaczenie	Przykład
Parametr	Nazwa zmiennej w definicji funkcji.	imię w function powitaj(imię)

Pojecie	Znaczenie	Przykład
Argument	Rzeczywista wartość przekazana przy wywołaniu.	"Ala" w powitaj("Ala")

15.3. Funkcja z dwoma argumentami

```
function dodaj(a, b) {
   let suma = a + b;
   console.log(suma);
}

dodaj(4, 6); // 10
```

Liczba argumentów i ich kolejność ma znaczenie. W wywołaniu dodaj(4, 6), parametr a otrzyma 4, a parametr b otrzyma 6.

15.4. Return - zwracanie wyniku

return kończy wykonywanie funkcji i przekazuje wartość z powrotem do miejsca wywołania. To pozwala zapisać wynik funkcji w zmiennej albo wykorzystac go w dalszym obliczeniu.

```
function dodaj(a, b) {
   return a + b;
}

let wynik = dodaj(4, 6);
console.log(wynik); // 10
```

console.log() a return

console.log() wyświetla wartość w konsoli. return przekazuje wartość z funkcji. Funkcja może wyświetlić wynik, ale go nie zwrócić. Gdy funkcja nie wykona return z wartością, jej wynikiem jest undefined.

15.5. Return kończy funkcje

```
function sprawdzWiek(wiek) {
   if (wiek < 18) {
      return "Niepelnoletni";
   }
   return "Pelnoletni";
}

console.log(sprawdzWiek(17));
```

Po wykonaniu return funkcja kończy się. Instrukcje umieszczone ponizej return w tej samej sciezce wykonania nie zostana uruchomione.

15.6. Funkcja jako sposób na podział programu

```
function poleProstokata(a, b) {
   return a * b;
}
```

```
function obwodProstokata(a, b) {
   return 2 * a + 2 * b;
}

let pole = poleProstokata(4, 5);
let obwod = obwodProstokata(4, 5);

console.log(pole);
console.log(obwod);
```

Dobrze zaprojektowana funkcja powinna mieć jedno czytelne zadanie. Nazwa funkcji powinna opisywac, co robi, np. obliczSume, sprawdzParzystosc, pobierzImie.

16. Rodzaje funkcji, arrow functions, zakres i callbacki

16.1. Deklaracja funkcji

```
function pomnoz(a, b) {
   return a * b;
}

console.log(pomnoz(3, 4));
```

Jest to klasyczny zapis function nazwa() { ... }. Funkcje deklarowane w ten sposób są czytelne i często wykorzystywane do opisywania osobnych czynnosci.

16.2. Function expression - funkcja przypisana do zmiennej

```
const pomnoz = function(a, b) {
   return a * b;
};

console.log(pomnoz(3, 4));
```

Funkcja anonimowa nie ma wlasnej nazwy po slowie function. Jest przypisana do zmiennej pomnoz, a wywołujemy ja przez nazwe tej zmiennej.

16.3. Funkcja strzalkowa (arrow function)

Funkcja strzalkowa to krótszy zapis funkcji wyrazonej przez =>. Często wykorzystywana jest jako callback w metodach tablicowych.

```
// Zapis klasyczny jako function expression
const dodaj1 = function(a, b) {
   return a + b;
};

// Zapis strzalkowy
const dodaj2 = (a, b) => {
   return a + b;
};

// Jeszcze krocej, gdy jest tylko jedno wyrazenie
const dodaj3 = (a, b) => a + b;
```

Postac	Zapis
Dwa lub więcej parametrów	(a, b) => a + b
Jeden parametr	a => a * 2
Brak parametrów	() => "Witaj"
Kilka instrukcji	(a, b) => { let suma = a + b; return suma; }

Kiedy można pominac return?

Gdy funkcja strzalkowa składa się z jednego wyrażenia bez nawiasów klamrowych, wynik tego wyrażenia jest zwracany automatycznie. Gdy uzywasz { }, musisz wpisać return, jeśli chcesz zwrócić wartość.

16.4. Parametry domyślne

Parametr może mieć wartość domyślna, która zostanie wykorzystana, gdy przy wywołaniu nie przekazano wartości albo przekazano undefined.

```
function powitaj(imie = "gosciu") {
   return `Czesc, ${imie}!`;
}

console.log(powitaj("Ala"));
console.log(powitaj());
```

16.5. Parametr rest - dowolna liczba argumentów

Zapis ...nazwa pozwala zebrac pozostale argumenty do tablicy. Parametr rest musi być ostatnim parametrem funkcji.

```
function suma(...liczby) {
   let wynik = 0;

   for (const liczba of liczby) {
      wynik += liczba;
   }

   return wynik;
}

console.log(suma(2, 4, 6)); // 12
```

16.6. Zakres zmiennych w funkcjach

Zmienna utworzona wewnątrz funkcji jest lokalna dla tej funkcji. Nie można jej użyć bezpośrednio poza jej zakresem. Parametry funkcji również zachowują się jak zmienne lokalne.

```
let stawka = 20; // globalna

function obliczWyplate(godziny) {
   let wyplata = godziny * stawka; // lokalna
   return wyplata;
}

console.log(obliczWyplate(8));
```

```
// console.log(wyplata); // blad - zmienna jest lokalna
```

16.7. Callback - funkcja przekazana do innej funkcji

Callback to funkcja przekazana jako argument innej funkcji. Metody tablicowe wykorzystuja callbacki, żeby okreslic, co zrobić z każdym elementem.

```
const liczby = [1, 2, 3];

function pokazElement(element) {
   console.log(element);
}

liczby.forEach(pokazElement);
```

Callback można zapisać jako nazwana funkcje albo funkcje anonimowa / strzalkowa. Funkcje nie wywołujemy w miejscu przekazania - przekazujemy ja, aby inna funkcja mogła wywołać ja w odpowiednim momencie.

17. Wyrażenia regularne - czym są i jak je zapisujemy

Wyrażenie regularne (RegExp, regex) to wzorzec opisujacy, jakiego fragmentu tekstu szukamy. Może sluzyc do wyszukiwania, sprawdzania poprawnosci danych i zamiany tekstu.

17.1. Do czego przydaje się regex?

- Sprawdzenie, czy tekst ma odpowiedni format, np. kod pocztowy 00-000.
- Wyszukanie wszystkich cyfr w tekście.
- Odnalezienie słów zaczynajacych się na wybrana litere.
- Usuniecie lub zamiana fragmentow tekstu.

17.2. Podstawowy zapis

```
/wzorzec/flagi
```

W JavaScript najczęściej zapisujemy regex pomiędzy ukośnikami. Pierwsza część opisuje wzorzec, a litery po drugim ukośniku to flagi zmieniające sposób dopasowania.

```
let tekst = "Odwiedz W3Schools";
let wzorzec = /w3schools/i;

console.log(wzorzec.test(tekst)); // true
```

W przykladzie flaga i oznacza ignorowanie wielkości liter, dlatego wzorzec dopasuje W3Schools mimo malych liter w wyrażeniu regularnym.

17.3. Znaki specjalne

Niektore znaki maja w regexie specjalne znaczenie. Gdy chcemy dopasowac taki znak doslownie, musimy poprzedzic go backslashem, czyli znakiem ucieczki \.

Znak	Znaczenie w regexie
^ / $	Początek i koniec tekstu (kotwice).
.	Dowolny pojedynczy znak, z wyjatkami opisanymi przez flage s.
[ ]	Klasa znaków - jeden znak z podanego zbioru.
( )	Grupa, która łączy elementy wzorca.
{ }	Liczba powtórzeń.
* + ?	Kwantyfikatory - ile razy element ma wystapic.
|	Alternatywa, czyli lub.
\	Znak ucieczki lub początek skroconej klasy, np. \d.

```
/./.test("A");      // true - kropka dopasowuje znak
/\./.test(".");    // true - \. dopasowuje doslowna kropke
/\./.test("A");    // false
```

Dosłowna kropka a dowolny znak

W regexie . oznacza dowolny znak. Jeśli szukasz prawdziwej kropki, użyj \. Przed backslashem w zwyklym stringu również trzeba pamietac o ucieczce, dlatego wygodnie jest zapisywac regex jako /wzorzec/.

18. RegExp - klasy znaków, kotwice, kwantyfikatory i grupy

18.1. Klasy znaków [ ]

Nawiasy kwadratowe oznaczaja, ze w danym miejscu pasowac ma jeden znak ze wskazanego zbioru. Można podac pojedyncze znaki albo zakres.

Wzorzec	Znaczenie	Przykład dopasowania
[abc]	Jeden znak: a, b lub c.	a, b, c
[^abc]	Jeden znak inny niz a, b, c.	d, 7, z
[a-z]	Mała litera od a do z.	m
[A-Z]	Wielka litera od A do Z.	M
[0-9]	Dowolna cyfra.	7
[a-zA-Z]	Mała albo wielka litera lacinska.	K, a

```
/[abc]/.test("kot"); // false - nie ma a, b ani c
/[abc]/.test("las"); // true - wystepuje a
/^[0-9]$/.test("7");  // true
```

18.2. Skrocone klasy znaków

Zapis	Znaczenie
\d	Cyfra od 0 do 9.
\D	Znak, który nie jest cyfra.
\w	Znak słowa: litera lacinska, cyfra albo podkreślenie _.
\W	Znak niepasujacy do \w.
\s	Biały znak, np. spacja, tabulator lub nowa linia.
\S	Znak, który nie jest bialym znakiem.
.	Dowolny znak (domyślnie nie obejmuje znaku końca linii).

```
/\d/.test("A7");      // true
/\d+/.test("ABC");    // false
/\s/.test("A B");     // true
/\w+/.test("abc_12"); // true
```

Uwaga na polskie litery i \w

Skrocone \w w podstawowym uzyciu jest nastawione na litery A-Z i a-z, cyfry oraz podkreślenie. Nie oznacza automatycznie wszystkich liter polskiego alfabetu. Przy bardziej zaawansowanej walidacji znaków narodowych warto poznac klasy Unicode i flage u.

18.3. Kotwice i pozycja dopasowania

Zapis	Znaczenie	Przykład
^	Początek tekstu (lub wiersza z flaga m).	^A - tekst zaczyna się od A.
$	Koniec tekstu (lub wiersza z flaga m).	z$ - tekst kończy się na z.
\b	Granica słowa - miejsce między znakiem słowa i znakiem niebedacym znakiem słowa.	\bkot\b - cale słowo kot.
\B	Miejsce, które nie jest granica słowa.	\Bkot - kot nie zaczyna się na granicy słowa.

```
/^A/.test("Ala");      // true
/t$/.test("kot");      // true
/\bkot\b/.test("mam kota"); // false - jest "kota", a nie "kot"
/\bkot\b/.test("kot dom");  // true
```

18.4. Kwantyfikatory - liczba powtórzeń

Zapis	Znaczenie	Przykład
*	Zero lub więcej wystąpień poprzedniego elementu.	ab*c pasuje do ac, abc, abbc.
+	Jedno lub więcej wystąpień.	ab+c pasuje do abc, abbc, ale nie ac.

Zapis	Znaczenie	Przykład
?	Zero lub jedno wystąpienie.	colou?r pasuje do color i colour.
{n}	Dokładnie n powtórzeń.	\d{3} - trzy cyfry.
{n,}	Co najmniej n powtórzeń.	\d{2,} - co najmniej dwie cyfry.
{n,m}	Od n do m powtórzeń.	\d{2,4} - od 2 do 4 cyfr.

```
/^\d{2}$/.test("45");     // true - dokladnie 2 cyfry
/^\d{2,4}$/.test("123");  // true - od 2 do 4 cyfr
/^a+$/.test("aaa");       // true - co najmniej jedna litera a
```

18.5. Grupy i alternatywa

Nawiasy okrągłe grupuja fragment wzorca. Znak | oznacza alternatywe, czyli dopasowanie jednej z możliwości.

```
/^(kot|pies)$/.test("kot");  // true
/^(kot|pies)$/.test("ryba"); // false

/^(ab){2}$/.test("abab");    // true
```

W pierwszym przykladzie caly tekst ma być równy kot albo pies. W drugim grupa ab ma powtórzyć się dokładnie dwa razy.

18.6. Znak ucieczki i kody znaków

Backslash zmienia znaczenie następnego znaku. Pozwala uruchomic skrócona klase, np. \d, ale także dopasowac dosłowny znak specjalny, np. \., \? albo \+ .

Zapis	Znaczenie
\.	Dosłowna kropka.
\?	Dosłowny znak zapytania.
\+	Doslowne plus.
\x20	Znak o podanym kodzie szesnastkowym (tu spacja).
\u0020	Znak Unicode o podanym kodzie (tu spacja).

19. Flagi i wykorzystywanie RegExp w metodach tekstowych

19.1. Flagi (modyfikatory)

Flagi zapisuje się po zamykajacym ukośniku wyrażenia regularnego. Zmieniaja sposób dopasowania wzorca.

Flaga	Działanie	Przykład zastosowania
i	Ignoruje wielkość liter.	/kot/i dopasuje kot, Kot i KOT.
g	Dopasowuje wszystkie wystąpienia, a nie tylko pierwsze.	/a/g szuka wszystkich liter a.

Flaga	Działanie	Przykład zastosowania
m	Zmienia działanie ^ i $ na poczatki i konce wierszy.	W tekście wieloliniowym.
s	Pozwala kropce . dopasowac także znak nowej linii.	Gdy wzorzec ma przechodzic przez kilka wierszy.
u	Włącza obsługę Unicode w regexie.	Przy bardziej zaawansowanych znakach Unicode.

```
let tekst = "Kot, kot i KOT";

console.log(tekst.match(/kot/));   // pierwsze dopasowanie
console.log(tekst.match(/kot/gi)); // wszystkie, bez rozrozniania wielkosci liter
```

19.2. Test() - sprawdzenie, czy wzorzec występuje

test() jest metoda obiektu RegExp. Zwraca true, jeśli w tekście jest dopasowanie, albo false, jeśli go nie ma.

```
let wzorzec = /\d{3}/;
console.log(wzorzec.test("Kod 123")); // true
console.log(wzorzec.test("Kod ABC")); // false
```

19.3. Exec() - pobranie dopasowania

exec() szuka dopasowania i zwraca obiekt z informacjami o dopasowaniu albo null, jeśli nic nie znaleziono. Przy fladze g można używać go wielokrotnie do pobierania kolejnych dopasowan.

```
let wzorzec = /\d+/;
let wynik = wzorzec.exec("Numer 12345");

console.log(wynik[0]); // "12345"
```

19.4. Metody String wspolpracujace z regex

Metoda	Co zwraca / robi?	Przykład
search(regex)	Indeks pierwszego dopasowania albo -1.	tekst.search(/\d/)
match(regex)	Dopasowanie albo tablice dopasowan (z flaga g).	tekst.match(/\d+/g)
replace(regex, tekst)	Zwraca tekst z zamienionym dopasowaniem.	tekst.replace(/kot/i, "pies")
replaceAll(regex, tekst)	Zamienia wszystkie wystąpienia; regex powinien mieć flage g.	tekst.replaceAll(/a/g, "o")
split(regex)	Dzieli tekst w miejscach pasujacych do wzorca.	tekst.split(/[,;]+/)

```
let tekst = "Ala ma 2 koty i 3 psy";

console.log(tekst.search(/\d/));
console.log(tekst.match(/\d+/g));
```

```
console.log(tekst.replace(/\d+/g, "X"));
```

19.5. Przykłady wzorcow z notatek

Zastosowanie	Wyrażenie regularne	Co sprawdza?
Kod pocztowy	^\d{2}-\d{3}$	Dwie cyfry, myślnik, trzy cyfry i nic więcej.
Prosty adres www	^www\..+\..+$	Tekst zaczyna się od www., ma dalszy fragment, kropkę i kolejny fragment.
Liczba rzeczywista z kropka	^-?\d+(\.\d+)?$	Opcjonalny minus, cyfry, opcjonalnie kropka i cyfry.

Wzorce z powyzszej tabeli są szkolnymi przykladami. Szczególnie adres www jest uproszczony i nie sprawdza wszystkich zasad poprawnego adresu internetowego. Regex do walidacji danych w prawdziwych formularzach wymaga dokladniejszego dopasowania do wymagan.

```
// Kod pocztowy
/^\d{2}-\d{3}$/.test("85-123"); // true

// Liczba: 15, -7, 3.14, -0.5
/^-?\d+(\.\d+)?$/.test("-3.14"); // true
/^-?\d+(\.\d+)?$/.test("3,14");  // false - wzorzec wymaga kropki
```

20. Przykłady złożone i typowe zadania

W tej części kilka tematów łączy się w calosc. Takie przykłady pokazuja, jak używać operatorow, pętli, tablic, stringow i funkcji razem.

20.1. Zliczenie liczb parzystych w tablicy

```
const liczby = [3, 8, 10, 5, 12, 7];
let ile = 0;

for (let i = 0; i < liczby.length; i++) {
   if (liczby[i] % 2 === 0) {
      ile++;
   }
}

console.log(ile); // 3
```

Pętla przechodzi po każdym indeksie. Warunek sprawdza, czy reszta z dzielenia przez 2 wynosi 0. Jeśli tak, zwiekszamy licznik ile.

20.2. To samo z filter()

```
const liczby = [3, 8, 10, 5, 12, 7];
const parzyste = liczby.filter(liczba => liczba % 2 === 0);

console.log(parzyste);       // [8, 10, 12]
console.log(parzyste.length); // 3
```

20.3. Suma elementow tablicy

```
const liczby = [4, 6, 10];
let suma = 0;

for (const liczba of liczby) {
   suma += liczba;
}

console.log(suma); // 20
```

WERSJA KROTSZA

```
// Wersja z reduce()
const suma2 = liczby.reduce((wynik, liczba) => wynik + liczba, 0);
console.log(suma2);
```

20.4. Rozdzielenie tekstu na wyrazy i zliczenie wyrazów

```
let zdanie = "JavaScript jest jezykiem stron";
let wyrazy = zdanie.split(" ");

console.log(wyrazy);
console.log(wyrazy.length); // 4
```

Jeśli tekst może mieć kilka spacji pod rzad, split(" ") utworzy też puste elementy. W takiej sytuacji można wykorzystac regex, który dzieli tekst po jednym lub wielu białych znakach.

```
let zdanie = "JavaScript   jest\njezykiem stron";
let wyrazy = zdanie.trim().split(/\s+/);
console.log(wyrazy);
```

20.5. Funkcja sprawdzajaca kod pocztowy

```
function poprawnyKodPocztowy(kod) {
   const wzorzec = /^\d{2}-\d{3}$/;
   return wzorzec.test(kod);
}

console.log(poprawnyKodPocztowy("85-123")); // true
console.log(poprawnyKodPocztowy("85123"));  // false
```

20.6. Losowanie napisu z liter

```
function losowyNapis(dlugosc) {
   let wynik = "";

   for (let i = 0; i < dlugosc; i++) {
      let kod = Math.floor(Math.random() * 26) + 65;
      wynik += String.fromCharCode(kod);
   }

   return wynik;
}
```

```
console.log(losowyNapis(8));
```

Kod znaku A to 65, a Z to 90. Losujemy liczbę całkowita z zakresu 65-90 i zamieniamy ja na znak. Funkcja powtarza to tyle razy, ile wynosi parametr długość.

20.7. Funkcja obliczajaca średnia

```
function obliczSrednia(oceny) {
  if (oceny.length === 0) {
    return 0;
  }

  const suma = oceny.reduce((wynik, ocena) => wynik + ocena, 0);
  return suma / oceny.length;
}

console.log(obliczSrednia([3, 4, 5])); // 4
```

Przed dzieleniem sprawdzamy, czy tablica nie jest pusta. Bez tego dzielenie sumy przez 0 nie daloby poprawnej średniej.

21. Najczęstsze błędy i szybka powtórka

21.1. Błędy, na które trzeba uważać

Błąd	Dlaczego jest niepoprawny?	Poprawna zasada
Użycie = zamiast === w warunku	= przypisuje wartość, a nie porownuje.	Do porównania uzywaj ===.
Pomijanie indeksu 0	Tablice licza indeksy od zera.	Pierwszy element to tablica[0].
tablica.length()	length jest właściwością, a nie metoda.	Zapisz tablica.length.
sort() dla liczb bez funkcji	Domyślnie sortuje jako tekst.	Użyj sort((a,b) => a-b).
Mylenie splice() i slice()	Jedna metoda zmienia tablice, druga zwraca fragment.	splice() modyfikuje, slice() kopiuje fragment.
Oczekiwanie, ze forEach zwróci nowa tablice	forEach() zwraca undefined.	Do nowej tablicy użyj map() albo filter().
Brak return w funkcji	Funkcja nie przekazuje wyniku.	Dodaj return, gdy wynik ma być zwrócony.
Oczekiwanie, ze console.log to return	Logowanie tylko wyświetla dane.	Użyj return do zwrocenia wartości.
Pominiecie break w switch	Wykonują się kolejne case.	Dodaj break, chyba ze chcesz fall-through.
Pętla bez zmiany licznika	Warunek może nigdy nie stac się fałszywy.	Aktualizuj licznik albo zmienna warunku.
tekst.sort()	String nie ma metody sort().	Zmień tekst na tablice: split().sort().join().
Użycie typeof do sprawdzenia tablicy	typeof [] zwraca "object".	Użyj Array.isArray().
Zalozenie, ze prompt zwraca liczbę	prompt() zwraca tekst albo null.	Przekonwertuj np. Number(prompt(...)).

21.2. Ściąga - co zwracają najważniejsze metody?

Element	Zwracana wartość / rezultat
push(x)	Nowa długość tablicy.
pop()	Usuniety ostatni element.
unshift(x)	Nowa długość tablicy.

Element	Zwracana wartość / rezultat
shift()	Usuniety pierwszy element.
splice()	Tablica usunietych elementow; jednocześnie zmienia oryginal.
slice()	Nowa tablica z wybranym fragmentem.
forEach()	undefined.
map()	Nowa tablica wynikow.
filter()	Nowa tablica wybranych elementow.
reduce()	Jedna wartość wynikowa.
find()	Pierwszy znaleziony element albo undefined.
some() / every()	true albo false.
indexOf()	Indeks albo -1.
includes()	true albo false.
search()	Indeks dopasowania albo -1.
match()	Dopasowanie albo null / tablica dopasowan zalezne od wzorca i flag.
test()	true albo false.
exec()	Obiekt dopasowania albo null.

21.3. Ściąga - co zmienia oryginal?

Zmienia tablice	Nie zmienia oryginalnej tablicy / tworzy wynik
push(), pop(), shift(), unshift()	slice(), concat(), map(), filter()
splice(), sort(), reverse()	toSorted(), toReversed(), toSpliced()

Metody String, takie jak toUpperCase(), trim(), replace() czy slice(), zwracają nowy tekst. Stringi są niemutowalne, dlatego do zmiany wartości zmiennej trzeba przypisać wynik.

22. Zestaw ćwiczeń z odpowiedziami

Najpierw spróbuj rozwiązać zadania samodzielnie. Odpowiedzi znajdziesz pod każdym zadaniem - zaslon je podczas pierwszej proby.

Zadanie 1. Typy danych

Podaj wynik i typ kazdego wyrażenia: 5 + 2, "5" + 2, "5" - 2, typeof true.

Odpowiedzi

5 + 2 daje 7 (number); "5" + 2 daje "52" (string); "5" - 2 daje 3 (number); typeof true daje tekst "boolean".

Zadanie 2. Porównania

Jaki wynik dadza wyrażenia: 5 == "5" oraz 5 === "5"?

Odpowiedzi

5 == "5" daje true, ponieważ operator == dopuszcza konwersje typu. 5 === "5" daje false, ponieważ typy są różne.

Zadanie 3. Pętla

Co wypisze pętla: for (let i = 1; i <= 4; i++) { console.log(i * 2); }?

Odpowiedzi

2, 4, 6, 8. Licznik przyjmuje wartości od 1 do 4, a kazda jest mnozona przez 2.

Zadanie 4. Tablica

Dla tablicy const liczby = [4, 8, 12, 16] podaj wynik liczby[2] i liczby.length.

Odpowiedzi

liczby[2] to 12, bo indeksy zaczynaja się od 0. liczby.length to 4, bo tablica zawiera cztery elementy.

Zadanie 5. Metody tablicowe

Czym roznia się map() i filter()?

Odpowiedzi

map() przekształca każdy element i zwraca nowa tablice wynikow. filter() wybiera tylko te elementy, dla których warunek zwróci true.

Zadanie 6. splice i slice

Która metoda zmienia oryginalna tablice: splice() czy slice()?

Odpowiedzi

splice() zmienia oryginalna tablice. slice() zwraca nowa tablice z wybranym fragmentem i nie zmienia oryginalu.

Zadanie 7. Funkcje

Co jest parametrem, a co argumentem w wywołaniu: obliczSume(4, 7), jeśli definicja to function obliczSume(a, b)?

Odpowiedzi

Parametry to a i b - nazwy w definicji funkcji. Argumenty to wartości 4 i 7 przekazane w wywołaniu.

Zadanie 8. RegExp

Co sprawdza wzorzec /^\d{2}-\d{3}$/?

Odpowiedzi

Caly tekst ma skladac się z dwóch cyfr, myślnika i trzech cyfr, bez dodatkowych znaków przed ani po wzorcu.

Zadanie 9. Napisz funkcje

Napisz funkcje, która otrzymuje tablice liczb i zwraca liczbę elementow parzystych.

```
function ileParzystych(liczby) {
   let licznik = 0;

   for (const liczba of liczby) {
      if (liczba % 2 === 0) {
         licznik++;
      }
   }

   return licznik;
}
```

Zadanie 10. RegExp

Napisz wzorzec, który sprawdzi, czy tekst składa się z dokładnie czterech cyfr.

Odpowiedź

Wzorzec: /^\d{4}$/. Kotwice ^ i $ wymagaja, aby tekst od początku do końca pasowal do wzorca.

23. Słowniczek najważniejszych pojęć

Pojecie	Znaczenie
Argument	Wartość przekazana do funkcji podczas jej wywołania.
Array	Tablica - uporzadkowany zbior elementow dostepnych przez indeksy.
Boolean	Typ logiczny: true albo false.
Callback	Funkcja przekazana jako argument innej funkcji.
Concatenation / konkatenacja	Laczenie łańcuchów znaków.
Expression / wyrażenie	Fragment kodu, który oblicza wartość.
Function / funkcja	Wydzielony fragment kodu możliwy do wielokrotnego wywołania.
Index / indeks	Numer pozycji elementu tablicy lub znaku w tekście; zaczyna się od 0.
Iteration / iteracja	Jedno przejscie pętli lub przetwarzania kolejnego elementu.
Method / metoda	Funkcja zwiazana z wartością lub obiektem, np. tablica.push().
Parameter / parametr	Nazwa zmiennej w definicji funkcji.
Regular expression / regex	Wzorzec opisujacy, jakiego tekstu szukamy.

Pojecie	Znaczenie
Return value / wartość zwracana	Wartość przekazana przez funkcje za pomoca return.
Scope / zakres	Miejsce, w ktorym zmienna jest dostępna.
String	Łańcuch znaków, czyli tekst.
Truthy / falsy	Wartości traktowane w warunku jak true albo false.
Undefined	Wartość oznaczajaca brak przypisanej wartości.
Unicode	Standard przypisujacy kody znakom wielu języków i symbolom.

24. Materiały i źródła

Podstawa notatki: tematy z przesłanego PDF-u z zeszytu. Wyjaśnienia i rozszerzenia sprawdzono glownie z tutorialem JavaScript w serwisie W3Schools. W3Schools zawiera przykłady do uruchomienia w przeglądarce (Try it Yourself), dlatego warto otwierac linki i samodzielnie zmieniać wartości w kodzie.

Najbardziej przydatne strony W3Schools:

- JavaScript Tutorial - caly kurs
- JavaScript Variables
- var, let and const
- JavaScript Scope
- JavaScript Data Types
- JavaScript Type Conversion
- JavaScript Operators
- JavaScript Bitwise Operations
- if...else
- switch
- for Loop
- while Loop
- Array Methods
- Array Iteration
- Array Sort
- JavaScript Strings
- String Methods
- Function Definitions
- Function Parameters
- Function Return
- Arrow Functions
- Regular Expressions
- RegExp Flags
- RegExp Patterns

Jak uczyć się z W3Schools?

Przeczytaj wyjaśnienie, uruchom przykład Try it Yourself i zmień jedna rzecz naraz: wartość zmiennej, operator, indeks albo warunek. Potem przewidz wynik przed uruchomieniem. Tak najszybciej zrozumiesz, jak działa dana metoda.